CONFIDENTIAL · SECURITY AUDIT

VERSION 5 · RE-REVIEW

Security Audit Tech for Palestine Website

techforpalestine.org · Cloudflare Pages & Workers

This is the **fifth-round re-review**, assessing patches applied after v4 (2026-04-09). The medium-severity proxy path-allowlist finding (P-1) has been resolved, bringing the total fixed count to **17 of 25 issues**. Three new findings were identified during this round — one stemming from an incomplete normalisation step in the P-1 fix, one from conflicting middleware files, and one from unrestricted header forwarding in the proxy. All critical and high-severity application-code findings remain resolved.

V1 ORIGINAL	V2 RE-REVIEW	V3 RE-REVIEW	V4 RE-REVIEW	V5 RE-REVIEW
2026-03-31	2026-04-02	2026-04-06	2026-04-09	2026-04-13

Executive Summary — v5 Re-review

Delta audit of patches applied after v4 (2026-04-09). P-1 resolved; three new findings introduced.

17	0	1	2	4	7
TOTAL FIXED	CRITICAL	HIGH	MEDIUM	LOW	OPEN

Remediation progress: 17 of 25 findings resolved (68%)

68%

All critical and high-severity application-code findings remain resolved. H-4 (rate limiting) stays open as an infrastructure-configuration item requiring no code changes. The v4 fix for P-1 was sound in intent — restricting the proxy to `/api/method/` paths — but the string-prefix check can be bypassed by URL dot-segment sequences, introducing a follow-on finding P-2. Two additional findings (M-6, L-5) were identified in the same proxy file.

COMPLETE ISSUE REGISTER — V5 STATUS

ID	Finding	Peak Severity	v5 Status
C-1	API key & account ID in wrangler configs	CRITICAL	Fixed v2
C-2	Unauthenticated membership invite endpoint	CRITICAL	Fixed v2
C-3	Unverified webhook triggers privileged invite	CRITICAL	Fixed v2
N-1	Webhook secret exposed in URL query param	CRITICAL	Fixed v3
N-2	Timing attack on token comparison	HIGH	Fixed v3
H-1	Secret key logged to browser console	HIGH	Fixed v3
H-2	Full donor PII payload echoed in HTTP response	HIGH	Fixed v4
H-3	Middleware caches POST/API responses as public	HIGH	Fixed v3
H-5	Wildcard CORS on all endpoints	HIGH	Fixed v3
N-3	membership-invite proxies arbitrary body fields	MEDIUM	Fixed v3
M-1	No CSRF / Origin check on form endpoints	MEDIUM	Fixed v4

ID	Finding	Peak Severity	v5 Status
M-2	No input format/length validation on forms	MEDIUM	Fixed v4
M-3	getEnv globalThis fallback	MEDIUM	Fixed v3
M-4	Donor PII (email) in server logs	MEDIUM	Fixed v4
M-5	Internal error.message in HTTP responses	MEDIUM	Fixed v3
R-1	PUBLIC_SECRET_KEY in client-side Axios bundle	MEDIUM	Fixed v4
R-2	CSP 'unsafe-inline' defeats XSS protection	MEDIUM	Fixed v4
P-1	project-proxy: Authorization forwarded to any upstream path	MEDIUM	Fixed v5 ✓
L-1	Notion DB IDs in wrangler.toml	LOW	Fixed v3
P-2	project-proxy: dot-segment traversal bypasses /api/method/ prefix check (NEW)	NEW MEDIUM	New
M-6	Dual middleware conflict — cache-control layer is dead code (NEW)	NEW MEDIUM	New
H-4	No rate limiting on any endpoint	HIGH	Open
L-2	QGiv embed script loaded without SRI hash	LOW	Open
L-3	Image proxy has no authentication	LOW	Open
L-4	style-src still uses 'unsafe-inline' in CSP	LOW	Open
L-5	project-proxy forwards all browser request headers to upstream (NEW)	NEW LOW	New

CONFIRMED FIXED SINCE V4

- **P-1 — project-proxy path allowlist added** (`src/pages/api/project-proxy.ts:29`): The path guard was tightened from checking only for a leading `/` to requiring the path to start with `/api/method/` . Requests to any other upstream path now return HTTP 403 before the Authorization header is added. This closes the original P-1 vector. See P-2 for a follow-on bypass introduced by this fix.

NEW FINDINGS INTRODUCED BY V5 PATCH & REVIEW

NEW MEDIUM P-2

project-proxy: Dot-Segment Traversal Bypasses /api/method/ Prefix Check

File: src/pages/api/project-proxy.ts:29–36 **OWASP:** A01:2021 – Broken Access Control

The v5 fix added a `startsWith("/api/method/")` guard, but the check is performed on the raw, un-normalised path string received from the query parameter. The WHATWG URL Standard specifies that `fetch()` parses its URL argument and normalises dot-segment sequences (`/../` , `/./`) before making the network request. A caller can therefore supply:

```
// Passes the prefix check (starts with /api/method/)...
path = "/api/method/../api/auth/admin"

// ...but fetch() normalises the URL to:
upstream = "{API_URL}/api/auth/admin"   // ← Authorization injected here
```

ATTACK SCENARIO

Any browser visitor crafts a request to the proxy with a dot-segment path:

```
fetch("/api/project-proxy?path=/api/method/../api/auth/listUsers")
  // startsWith("/api/method/")  → passes guard ✓
  // fetch() resolves to: GET {API_URL}/api/auth/listUsers
  //   with Authorization: {SECRET_KEY}   ← full authenticated access
```

SECURE FIX — NORMALISE PATH BEFORE PREFIX CHECK

```
// project-proxy.ts — add URL normalisation before the startsWith guard
const raw = url.searchParams.get("path");

// Resolve dot-segments using the URL parser (same logic fetch() will use)
const normalised = raw ? new URL(raw, "http://localhost").pathname : "";

if (!normalised || !normalised.startsWith("/api/method/")) {
  return new Response(JSON.stringify({ error: "Path not allowed" }), {
    status: 403,
    headers: { "Content-Type": "application/json" },
  });
}

// Use the normalised path for the upstream URL
const upstream = `${apiUrl.replace(/\/$/, "")}${normalised}`;
```

NEW MEDIUM M-6

Dual Middleware Files — Cache-Control Layer Is Dead Code

Files: src/middleware.ts & src/middleware/index.ts **OWASP:** A05:2021 – Security Misconfiguration

Two separate files both export an `onRequest` middleware handler. Astro resolves the middleware entry point from a single location — `src/middleware.ts` takes precedence over `src/middleware/index.ts` per Vite's module resolution order. The CSP-nonce middleware in `src/middleware.ts` is therefore the only one that executes; the cache-control middleware in `src/middleware/index.ts` is silently never loaded.

```
// src/middleware/index.ts — NEVER EXECUTES in production
response.headers.set(
  "Cache-Control",
  !isGet || isApi ? "no-store" : "public, max-age=600"
);
```

Current exposure: all existing API routes set their own `Cache-Control: no-cache / no-store` headers explicitly, so there is no immediate gap. However, the security guarantee that "all API responses are never cached" rests entirely on individual route discipline — the middleware safety net the developer intends is not active. Any future route that omits a `Cache-Control` header will silently be cacheable.

FIX — COMBINE MIDDLEWARES USING ASTRO'S SEQUENCE()

```
// src/middleware.ts — unified entry point
import { defineMiddleware, sequence } from "astro:middleware";

const cacheMiddleware = defineMiddleware(async (context, next) => {
  const response = await next();
  const isApi = context.url.pathname.startsWith("/api/");
  const isGet = context.request.method === "GET";
  response.headers.set("Cache-Control", !isGet || isApi ? "no-store" : "public, max
-age=600");
  return response;
});

const cspMiddleware = defineMiddleware(async (context, next) => {
  // ... existing nonce logic from src/middleware.ts ...
});

export const onRequest = sequence(cacheMiddleware, cspMiddleware);
// Then delete src/middleware/index.ts
```

NEW LOW L-5

project-proxy Forwards All Browser Request Headers to Upstream API

File: src/pages/api/project-proxy.ts:38–41 **OWASP:** A01:2021 – Broken Access Control

The proxy copies the entire incoming browser request header set to the upstream fetch, deleting only `host`. Browser callers can therefore inject arbitrary headers into authenticated upstream requests:

```
// project-proxy.ts:38-41 — all browser headers forwarded
const headers = new Headers(request.headers);
headers.set("Authorization", secretKey);
headers.delete("host");   // only this one is stripped
// Cookie, X-Forwarded-For, X-Real-IP, CF-Connecting-IP, etc. still forwarded
```

A caller who knows the upstream API can inject `X-Forwarded-For` to spoof apparent client IP (relevant if the upstream uses IP-based access controls), forward session `Cookie` headers, or send custom upstream-specific headers that influence routing or privilege checks.

FIX — BUILD AN EXPLICIT FORWARDED-HEADER ALLOWLIST

```
// project-proxy.ts — replace header copy block
const FORWARD_HEADERS = ["content-type", "accept", "accept-language", "accept-encod
ing"];
const headers = new Headers();
for (const name of FORWARD_HEADERS) {
  const val = request.headers.get(name);
  if (val) headers.set(name, val);
}
headers.set("Authorization", secretKey);
```

REMAINING OPEN FINDINGS FROM PRIOR ROUNDS

HIGH H-4 No Rate Limiting on Any Endpoint — Open Since v1

File: All src/pages/api/* endpoints **OWASP:** A04:2021 – Insecure Design

Note: Configuration-level fix only — no code changes needed

Every write endpoint — creating Notion records, triggering Hub invitations, subscribing email addresses — accepts unlimited requests per IP. An attacker can flood `/api/endorsement-request` or `/api/e4p-pledge-sign` at full throughput to fill Notion databases with spam entries or exhaust API quotas. This has been the last remaining high-severity finding since v3 and requires only a Cloudflare dashboard configuration.

FIX — CLOUDFLARE WAF RATE LIMITING (FREE TIER, ~10 MIN TO CONFIGURE)

```
Cloudflare Dashboard → Security → WAF → Rate Limiting Rules

Rule 1 — Write endpoints (Notion forms):
  Expression : (http.request.method eq "POST")
               and starts_with(http.request.uri.path, "/api/")
  Action     : Block
  Threshold  : 10 requests / 60 seconds / IP

Rule 2 — Webhook (should only receive QGiv server traffic):
  Expression : starts_with(http.request.uri.path, "/api/qgiv-webhook")
  Action     : Block
  Threshold  : 30 requests / 60 seconds / IP
```

For the webhook, additionally consider configuring Cloudflare's IP allowlist to accept requests only from QGiv's published server IP ranges.

LOW L-2 QGiv Payment Embed Script Loaded Without Subresource Integrity

File: src/components/MembershipPage.tsx **OWASP:** A08:2021 – Software & Data Integrity Failures

The QGiv embed script (`https://secure.qgiv.com/resources/core/js/embed.js`) is dynamically injected into the DOM without an `integrity` attribute. A supply-chain compromise of QGiv's CDN could deliver malicious JavaScript into the payment context. Compute a SHA-384 hash of the known-good script version and add it as the `integrity` attribute.

```
// Compute: openssl dgst -sha384 -binary embed.js | openssl base64 -A
script.integrity = "sha384-<computed-hash>";
script.crossOrigin = "anonymous";
```

LOW L-3 Notion Image Proxy Has No Authentication

File: cloudflare-worker/worker.js **OWASP:** A01:2021 – Broken Access Control

The worker validates decoded URLs against known Notion S3 domain patterns, preventing open-relay abuse. Any actor with a valid Notion S3 URL can however use the proxy to cache and serve that image indefinitely. Low severity since the images are already public via Notion's own CDN. No immediate action required — a shared token header check would provide defence-in-depth if ever desired.

LOW L-4 style-src Still Uses 'unsafe-inline' in CSP

File: src/middleware.ts:27 **OWASP:** A05:2021 – Security Misconfiguration

`script-src` is correctly protected by per-request nonces. However, `style-src 'unsafe-inline'` remains. CSS injection is lower risk than script injection, but in some browser versions injected styles can be used to exfiltrate data via `background-image: url()` on attribute selectors. The primary fix is to move inline styles to external stylesheets or extend the HTMLRewriter to also nonce `<style>` tags.

```
// middleware.ts:27 — residual unsafe-inline
"style-src 'self' 'unsafe-inline' https://fonts.googleapis.com",
```

SECURITY POSTURE PROGRESS — ALL ROUNDS

Round	Date	Critical	High	Medium	Low	Total Open	Fixed
v1	2026-03-31	3	5	6	3	17	—
v2	2026-04-02	1	6	7	3	17	3
v3	2026-04-06	0	2	5	3	10	10
v4	2026-04-09	0	1	2	3	6	16
v5	**2026-04-13**	**0**	**1**	**2**	**4**	**7**	**17**

PRIORITISED REMEDIATION PLAN — V5

Priority	Action	ID	Effort
Immediate	Add URL normalisation (`new URL(path, "http://localhost").pathname`) before the `/api/method/` prefix check in `project-proxy.ts`	P-2	3 lines of code
Day 1	Combine both middleware files using `sequence()` in `src/middleware.ts` ; delete `src/middleware/index.ts` to restore cache-control enforcement	M-6	~20 min refactor
Day 1	Replace `new Headers(request.headers)` with an explicit allowlist of forwarded headers (`content-type` , `accept` , etc.) in `project-proxy.ts`	L-5	~10 lines of code
Week 1	Configure Cloudflare WAF rate limiting on POST endpoints and the webhook route	H-4	10 min (dashboard only)
Week 1	Add SRI `integrity` hash to QGiv embed script injection in `MembershipPage.tsx`	L-2	30 min
Week 1	Replace `style-src 'unsafe-inline'` with nonce or hash (extend HTMLRewriter to `<style>` tags or move inline styles to external sheets)	L-4	2–4 hours
Backlog	Consider adding a shared token to image proxy for defence-in-depth	L-3	1 hour

GOOD PRACTICES CONFIRMED IN V5 CODEBASE

- **Constant-time comparison used consistently:** All three secret comparisons (qgiv webhook, membership invite, project proxy is key-injected not compared) use `constantTimeEqual()` backed by Node.js `timingSafeEqual` — no timing oracle risk anywhere in the auth layer.
- **CSP nonce implementation is correct and complete for scripts:** Per-request nonce via `crypto.randomUUID()` ; Cloudflare `HTMLRewriter` injects nonce into all `<script>` tags; `strict-dynamic` propagates trust to dynamically loaded scripts; set on HTML responses only, not API routes.
- **Email PII protected in logs:** All `console.log` / `console.warn` / `console.error` calls involving email addresses in `qgiv-webhook.ts` log only `[redacted]@domain.com` (domain portion only), preventing PII exposure in Cloudflare Workers log streams.
- **Input validation complete on all form endpoints:** `endorsement-request.ts` and `e4p-pledge-sign.ts` both validate Origin header, email format (regex), URL format (`new URL()`), field presence, and field length (≤ 2 000 chars) before any Notion API call is made.
- **Server-side proxy cleanly keeps secret out of browser bundle:** Client `api.ts` carries no credentials; the proxy adds `Authorization` server-side only; hop-by-hop headers (`transfer-encoding` , `connection`) are stripped from upstream responses before forwarding.
- **wrangler.toml contains no secrets:** All credentials (Notion secret, Hub API key, QGiv webhook secret, EO API key) are configured exclusively through the Cloudflare Pages dashboard environment variables. The committed `.env.example` correctly shows empty values only.